Optimizing Flight Purchase Timing

STA 141B Final Project Report

Yun Jiang
Chen Liu
Jiayang Liu
Chuyuan Wang

Prepared for:
Prof. Nicolai Amann

Department of Statistics
University of California, Davis

December 12, 2025

1. Abstract

Air ticket price is affected by market, demand, and operating costs. Our research studies these elements by integrating of historical traffic data (2018–2024) with real-time prices on the Amadeus Flight Offers API. To understand how air ticket prices change, we used two methods: a basic OLS regression to measure economic relationships, and a Random Forest model to capture complicated patterns. There are some insights we found out. First, the number of passengers plays a very important role. More travelers generally make the prices higher, but at the same time, airlines also benefit in an economic way, which averages the cost for each ticket.

Second, airlines also follow different pricing strategies. In particular, low-cost airlines such as Spirit tend to set the price of their tickets average of almost $100 less than the major airlines. Third, when we looked at the data over time, we saw that different types of routes change differently after the pandemic. Most leisure routes have gone back to their pre-COVID patterns, while business routes are still running unusually high.

To visualize these results, we also built an interactive dashboard where people can see the long-term trends, seasonal changes, and see how the competitions between airlines affect prices on specific routes.

2. Introduction

2.1. Related Work

Over years, researchers have been exploring how the structure of airline markets shapes the prices of the air ticket. Another well-known concept is "price dispersion," where passengers on the same flight wind up paying very different fares. Borenstein and Rose (1994) found that this kind of price discrimination actually tends to be more common on competitive routes than in monopoly markets [1].

In our work, we build on these ideas by blending classic econometric techniques (like OLS) with newer machine learning tools (such as Random Forest). Our goal is to see if the old theories about market structure, pricing power, and price discrimination still hold up in the post-pandemic world.

2.2. Motivation and Problem Statement

Ticket prices depend on a complex mix of factors such as shifting demand, fuel costs, competition among airlines, and their complicated algorithms to manage revenue. Of course, there are some best 'rules of thumb' such as book two months ahead of time, but usually, these simple tricks grossly over-simplify the dynamic air market.

This project pins these patterns down for real, combining long-term historical data with real-time tracking. Unlike travel apps that give you vague predictions without showing their work, we're using clear statistics to show exactly how seasonality, market competition, and booking timing actually affect the price you pay.

2.3. Dataset and Variables

2.3.1. DB1B Market Dataset (Historical)

"The Airline Origin and Destination Survey" or DB1B is a 10% sample of airline tickets from carriers who report, and it is made available by the U.S. Department of Transportation via bts_data. We chose seven years' worth of data (between 2018 and 2024) for analysis to examine long-term changes. Variables include:

- Market Fare: The average cost per ticket.
- PASSENGER VOLUME: The number of passengers on a given route.

- Route Distance: Non-stop distance in miles.
- Carrier Identity: The reporting and operating airline codes.

To facilitate the daily analysis of the quarterly data set, we adopted a strategy called **simulated transaction date** to randomly assign dates in a given quarter to obtain a more realistic distribution.

2.3.2. Flight Prices Dataset

We supplemented our analysis with the Flight Prices dataset [2] to investigate specific pricing determinants. This dataset provides granular details on ticket costs relative to booking parameters. Key features used for modeling include:

- **Ticket Price:** The target variable for our regression models.
- **Days Left:** The number of days between booking and departure (lead time).
- **Duration:** Total flight time in hours.
- **Airline:** The specific carrier, allowing for carrier-specific price sensitivity analysis.

2.3.3. Real-time Validation (Amadeus API)

To ensure that historical results were valid under today's market dynamics, we assembled an online data set via the **Amadeus Flight Offers Search API** [3]. A high-quality, business-grade data feed, it enabled us to search online market prices for prime paths (such as DEN-JFK and DFW-ORD flights) within a 330-day look-back window. It directly accesses Global Distribution System market prices, ensuring the accuracy and trustworthiness.

2.4. Hypotheses and Assumptions

We hypothesize that:

1. **Competition**: The more concentrated the market (the higher the HHI), the higher average price will be because there will be less pressure from competition.
2. **Seasonality**: It follows a Fourier pattern irrespective of overall economic trends.
3. **Structural Drivers**: There are operational factors with implications for prices, namely that prices should be positively related to distance because of fuel and labor costs and negatively related to passenger volume, reflecting economies of scale and airplane productivity. We assume for our analysis that the 10% DB1B sample size is a statistically valid representation of the larger market and that the seasonal influences will remain relatively intact despite the market changes brought about by the pandemic.

3. Methods

3.1. Data Acquisition and Preprocessing

To overcome the limitations of having a sample size of only 10%, in the DB1B dataset, we significantly expanded our feature space by integrating external high-dimensional datasets sourced from Kaggle-a U.S. Airline Traffic & Flight Prices amounting to more than **31 GB** of raw data. This huge dataset allows the model to do robust **feature learning**, capturing subtle patterns in traffic density and pricing fluctuations that would elude smaller datasets.

The raw data were quite messy and required rigorous cleaning in order to assure validity of the analytics. We standardized column definitions across sources first, to allow consistent schema mapping. The critical handling of missing values in the field of `TkCarrier`, or Ticket Carrier, null imputation using the `OPERATING_CARRIER` field was performed to retain carrier identity for market share analysis.

To assess the direct relationship between route characteristics and price, we filtered the dataset to only nonstop itineraries (`MktCoupons = 1`). We also worked through severe data quality issues when initially exploring the data, where unrealistic fare values of over $70,000 were present. These likely represent data entry mistakes, and we systematically removed them using the Interquartile Range (IQR) method. Fares falling below the first quartile by more than $1.5 \times$ the IQR, or falling above the third quartile by more than $1.5 \times$ the IQR, were excluded. We see in Figure 1 how this leaves us with a clean, statistically sound distribution of fares across all major routes.

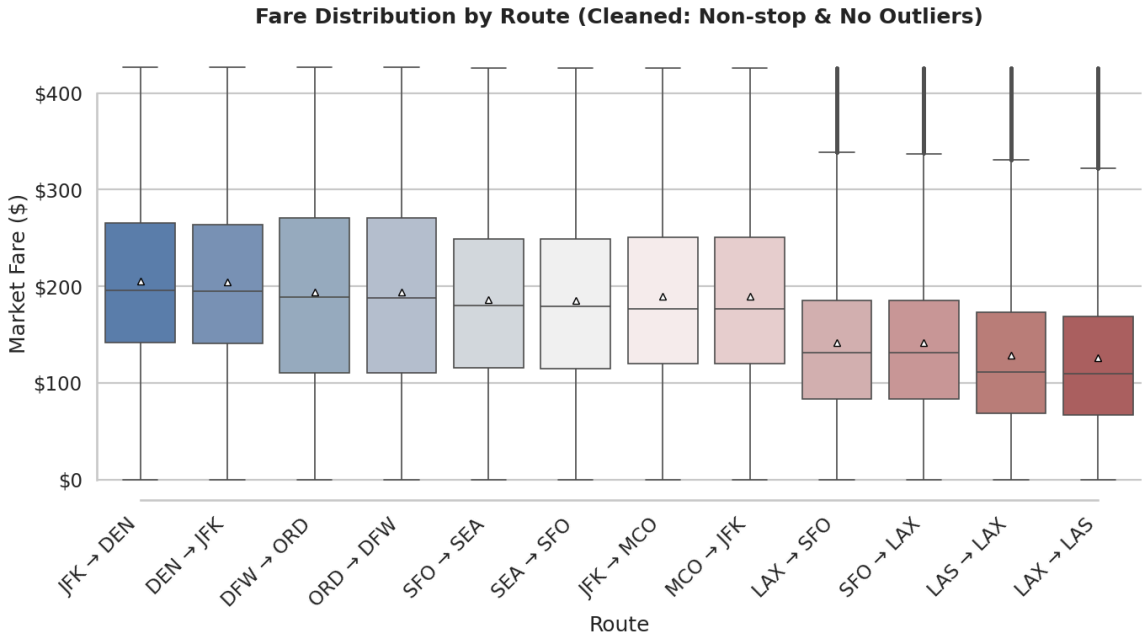


Figure 1: Distribution of Non-stop Fares by Route after cleaning

3.2. Predictive Modeling Strategy

In order to accurately determine the driving factors of air ticket prices, we have implemented a complementary Multi-Model Approach:

1. Multivariable Linear Regression (OLS): We first use OLS regression to establish a baseline and quantify the linear relationship between variables (e.g., the specific amount of fare increase per mile). This helps to explain the direction and statistical significance of each factor.
2. Random Forest Regressor: To capture complex non-linear interactions (e.g. changes in the influence of peak holidays and off-peak distance), we trained a Random Forest model. This non-parametric integration method can achieve high-precision feature importance sorting and identify key drivers that may be underestimated by linear models, such as Market Concentration (HHI).

3.3. Trend and Seasonality Modeling

We have used two different modeling approaches to forecast the temporal behavior of flight prices:

1. **Fourier Series Regression.** To capture the cyclical seasonal patterns, such as peaks in summer and troughs in winter, we performed linear regression extended with first-order Fourier terms. We created sine and cosine features based on a 365.25-day annual cycle. The model specification is:

$$\text{Fare}_t = \beta_0 + \beta_1 t + \beta_2 \sin\left(\frac{2\pi t}{365.25}\right) + \beta_3 \cos\left(\frac{2\pi t}{365.25}\right) + \varepsilon$$

where t represents the time index in days. This allowed us to decompose the price into a long-term linear trend and an oscillating seasonal component.

1. **SARIMA Forecasting**: To make a stricter time series forecast, we summarized the data into a quarterly average ticket price and applied the Seasonal AutoRegressive Moving Average (SARIMA) model. We utilized a standard quarterly seasonal order $(P, D, Q, s) = (1, 1, 1, 4)$ to account for the four-quarter cycle inherent in airline demand. The model captures the autocorrelation in pricing (how past prices affect future prices) and generates probability predictions with confidence intervals.

3.4. Short-Term Booking Window Modeling (SerpAPI Data)

To understand how air ticket prices change as the departure dates, we analyzed daily prices on AmadeusAPI. Since different routes have different baselines, all prices should be normalized so that the patterns of routes can be compared on the same level. The normalization uses each route's median fare from 60–90 days before departure:

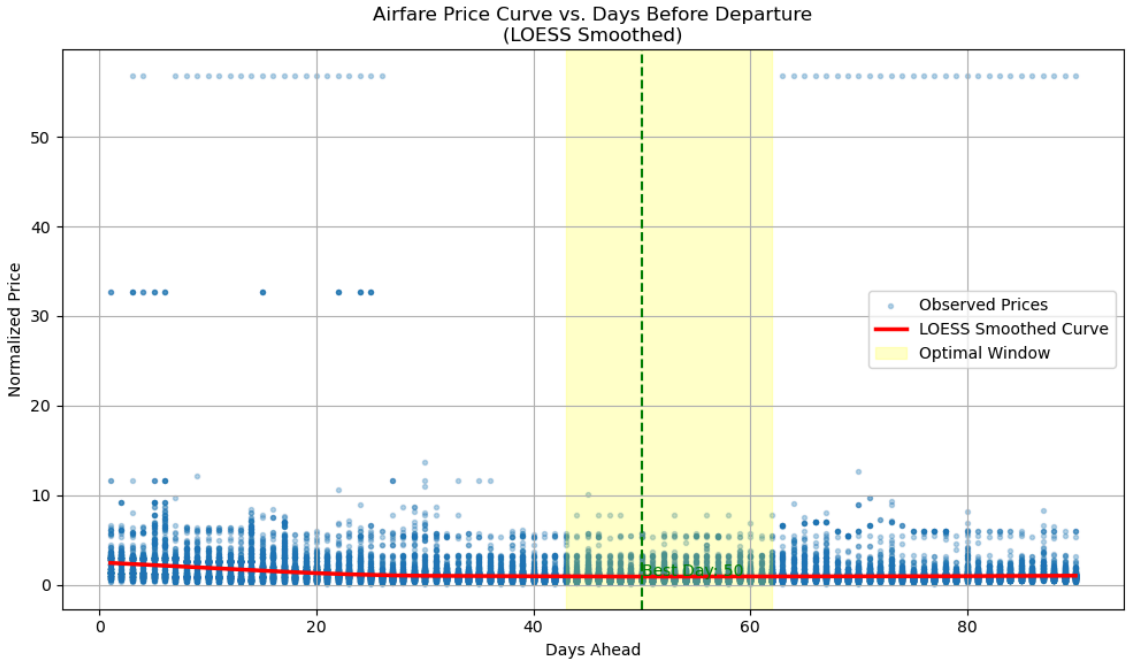
$$\tilde{P}(d) = \frac{P_{\text{observed}(d)}}{\text{median}\{P(d') : 60 \leq d' \leq 90\}}$$

Prices decrease gradually at the long horizons, reach their lowest point around 3–6 weeks before departure, and then rise during the final week.

According to this curve, the optimal booking window is between 21 and 45 days before departure, where fares are lowest. However, last-minute purchases (0–7 days out) are obviously more expensive.

Our results are:

- Best single day to buy: 50.0 days before departure
- Minimum normalized price: 0.938
- Optimal Booking Window: 43-62 days ahead
- Last-minute premium (0-7 days): 126.5%



3.5. Price Driver and Competition Analysis

Ordinary Least Squares (OLS) multiple regression and Random Forest regression were used to identify the variables most influential in determining ticket prices.

- **Market Concentration (HHI)**: We calculated the Herfindahl–Hirschman Index (HHI) for each route as an indicator of how competitive the airline market is on that route. The HHI is computed as the sum of the squared market shares of all carriers operating on a route. Using this measure, routes are classified as competitive (HHI < 1500), moderately concentrated (1500–2500), or highly concentrated (above 2500).
- **Multivariate Regression(OLS)**: We estimated a multivariable OLS regression with MktFare as the outcome to estimate how route and market factors relate to airfare, holding other variables constant. Predictors include MktDistance, Passengers, airline and quarter fixed effects, and a COVID dummy (2020–2021), with optional controls (load factor, booking days, basic economy ratio) and HHI_Index when available. Coefficients represent the conditional marginal change in MktFare per unit change in each predictor.
- **Machine Learning Feature Importance:** We trained a Random Forest Regressor to predict market fare based on features including market distance, passenger volume, HHI, COVID-19 indicators, and airline identifiers. Random Forest provides a non-linear analysis of feature importance, identifying which variables have the strongest predictive power for ticket prices.

4. Results

4.1. Historical Price Trends and Seasonality

The historical data shows a huge drop in both prices and demand from 2020 to 2022, reflecting the impact of the COVID-19 pandemic.

As shown in Figure 3, ticket prices across all major routes fell dramatically in early 2020. However, the recovery paths differ across routes. Business-heavy routes such as **DFW–ORD** (orange line), **DEN–JFK** (green line), and **SEA–SFO** (yellow line) rebounded more quickly after the pandemic. By contrast, leisure routes like **LAS–LAX** (pink line) experienced a slower, more modest recovery, stabilizing around post-pandemic levels rather than exceeding them.

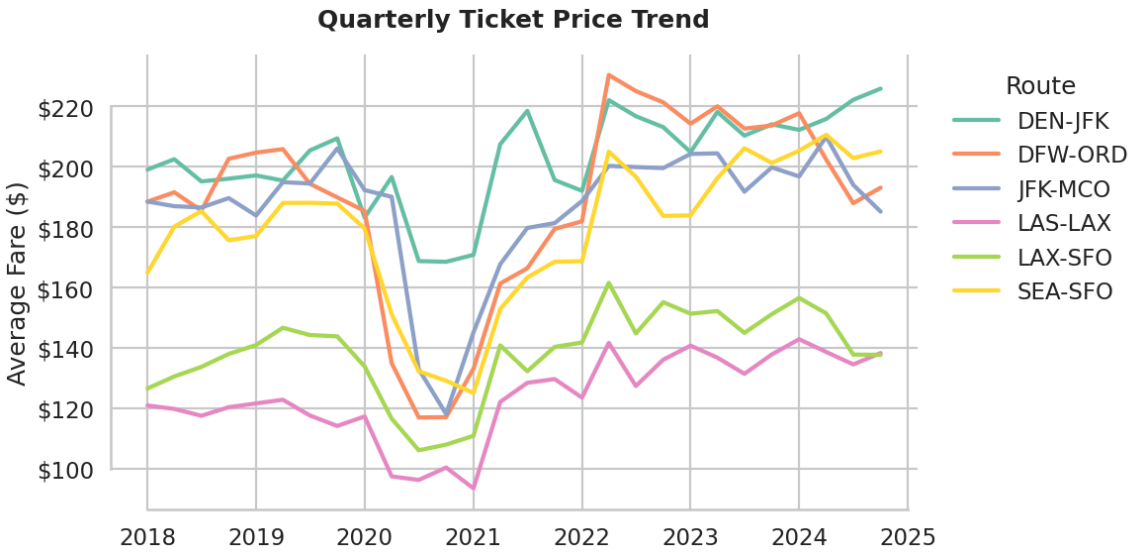


Figure 3: Quarterly Average Ticket Price Trend (2018–2024). Note the distinct V-shaped recovery and the inflationary "overshoot" on business routes (e.g., DFW–ORD) compared to leisure routes.

The trends in passenger flows (Figure 4) mirror those of prices but, on average, show relatively consistent demand. Despite the sharp drop in early 2020, passenger numbers have steadily recovered. The seasonal oscillations shown in the graph indicate a cyclical pattern of demand, which holds for both the pre-pandemic and post-pandemic periods.

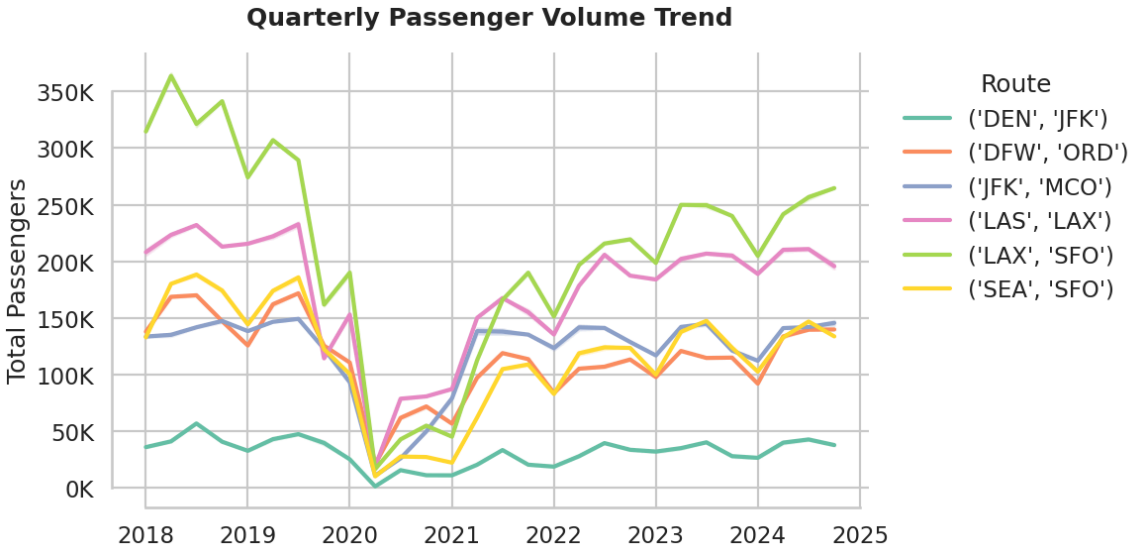


Figure 4: Quarterly Passenger Volume Trend. The consistent seasonal peaks and troughs indicate that travel demand seasonality has remained robust despite long-term economic shifts.

4.2. Route-Specific Dynamics: Leisure vs. Business

Building on the aggregate trends discussed above, we turn to route-level analyses to illustrate how these recovery patterns differ between leisure and business markets.

As shown in the 90-day rolling average trend Figure 5, the leisure-heavy **LAX to LAS** route exhibits a clear "return to baseline" pattern. Prices remained stable until early 2020, dropped sharply during the onset of the pandemic, and gradually recovered to pre-pandemic levels by 2023 without significant overshoot.

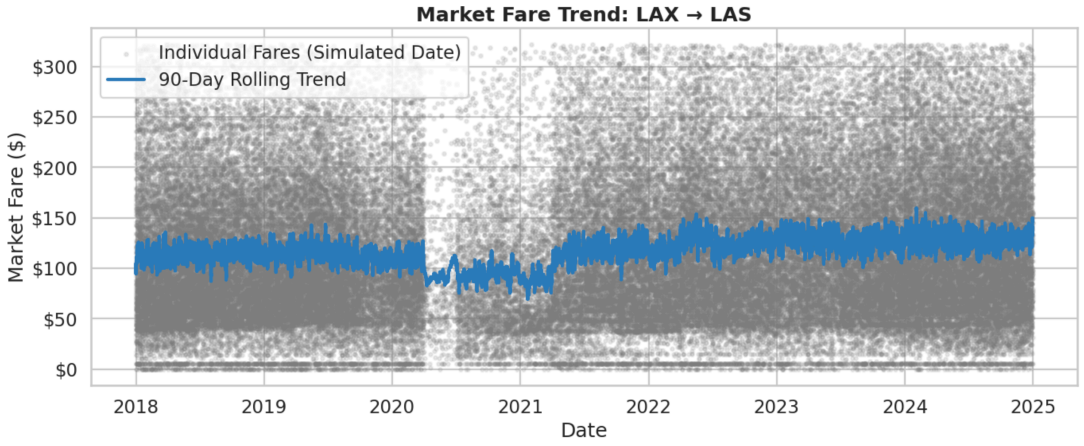


Figure 5: 90-Day rolling average fare trend for **LAX to LAS** (2018–2024). The blue trend line illustrates a U-shaped recovery where current prices have stabilized near their 2019 historical averages.

In contrast, the **ORD to DFW** route (Figure 6), a major business corridor, displays a stronger inflationary trend. Following the pandemic recovery, average fares in 2022–2023 surged significantly

higher than their 2019 peaks, indicating strong demand and inflationary pressures in this market segment.

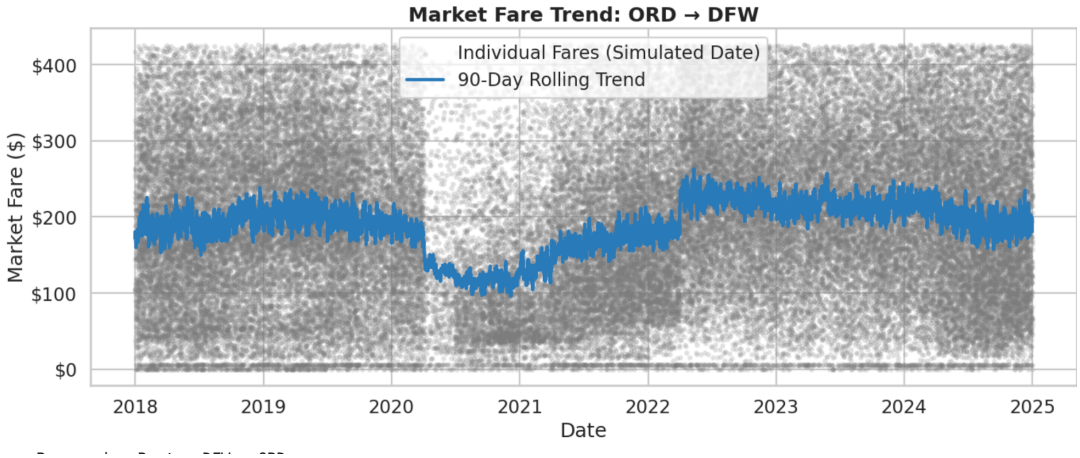


Figure 6: 90-Day rolling average fare trend for **ORD to DFW** (2018–2024). Note the distinct "price surge" post-2021, where the recovery trend (blue line) surpassed pre-pandemic price levels.

4.3. Market Concentration and Competition

The analysis of market structure using the HHI index reveals that the domestic routes analyzed are predominantly **highly concentrated**. The distribution of HHI scores shows a mean value of approximately 4002 and a median of 4380. Given that an HHI above 2500 is considered "highly concentrated" by regulatory standards, this suggests that most major routes are dominated by a small number of carriers.

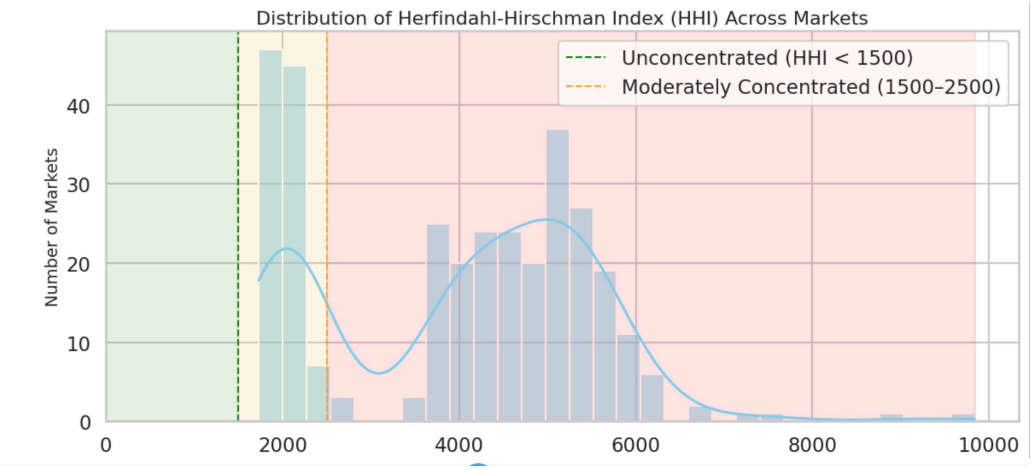


Figure 7: Distribution of Herfindahl-Hirschman Index (HHI) scores

4.4. Determinants of Airfare

To isolate the drivers of ticket prices, we used a dual-model approach: OLS Regression to quantify the specific dollar impact of each factor, and Random Forest to rank their overall predictive power.

4.4.1. Linear Drivers (OLS Regression)

The OLS model provides precise quantification of how each factor impacts ticket price in dollar terms relative to a baseline. In our model, **American Airlines (AA)** serves as the reference category. As detailed in Table 1, all key variables are statistically significant ($P < 0.001$).

Variable	Coefficient	Std. Error	P-Value
Intercept (Baseline: AA)	81.42	7.025	< 0.01
Spirit Airlines (NK)	−97.22	0.549	< 0.01
United Airlines (UA)	+1.65	0.319	< 0.01
Pandemic (Is_Covid)	−15.97	0.292	< 0.01
Passenger Volume	−0.93	0.007	< 0.01
Market Dist (Miles)	+0.03	0.001	< 0.01
Concentration (HHI)	+0.01	0.000	< 0.01

Table 1: OLS Regression Results

Key findings from the regression include:

- **Carrier Effects:** Pricing strategies vary significantly by airline. Compared to the baseline **American Airlines**, ultra-low-cost carrier **Spirit Airlines (NK)** trades at a massive discount, reducing the average fare by **$97.22**. In contrast, legacy competitor **United Airlines (UA)** commands a slight premium (+$1.65) over American.
- **Market Structure**: The `HHI_Index` coefficient is positive (+0.01), confirming that higher market concentration leads to higher fares. For every 100-point increase in HHI, average fares rise by approximately $1.
- **Economies of Scale**: `Passengers` shows a strong negative coefficient (−0.93), implying that adding passenger capacity correlates with lower per-ticket costs due to efficiency gains.

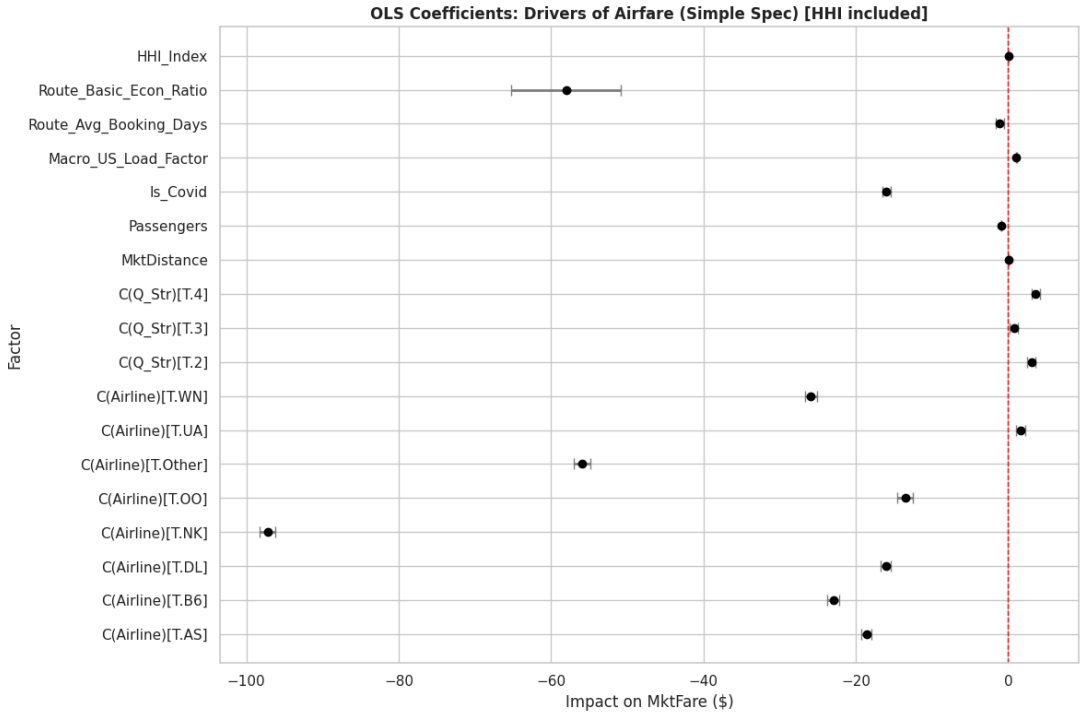


Figure 8: Visualizing the OLS Coefficients (95% CI). The red line at 0 represents the baseline (American Airlines). Factors to the left (like NK) indicate lower prices relative to AA, while factors to the right indicate premiums.

4.4.2. Non-Linear Importance (Random Forest)

While OLS explains the **direction** (positive/negative) of these relationships, the Random Forest model ranks their **predictive power**.

As shown in Figure 9, the model identified Passenger Volume as the single most informative feature (Importance ≈ 0.23), followed closely by Market Concentration (HHI) (Importance ≈ 0.18) and Flight Distance (0.18). This ranking refines our understanding: while market competition (HHI) is critical, the sheer scale of demand (Passenger Volume) is the dominant force in determining route-level pricing structures.

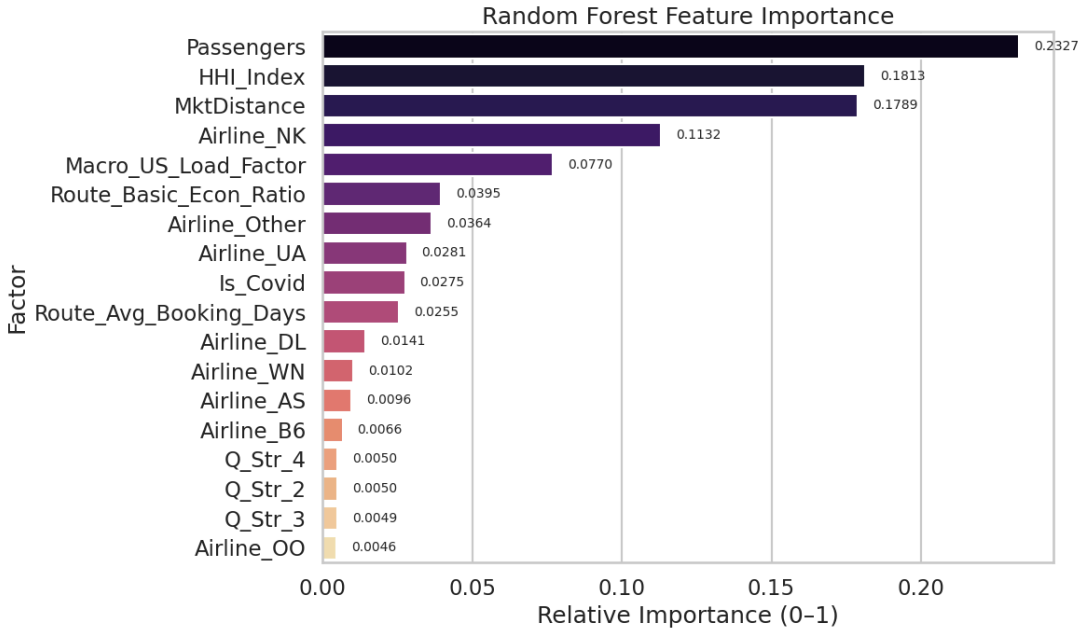


Figure 9: Random Forest Feature Importance

4.5. Forecasting

The SARIMA $(1, 0, 1) \times (1, 1, 1, 4)$ model provided a stable short-term forecast for route-specific pricing. For the JFK-MCO route, the model predicts a stabilization of prices over the next four quarters, with the forecasted values remaining within the historical variance observed in the post-pandemic recovery phase.

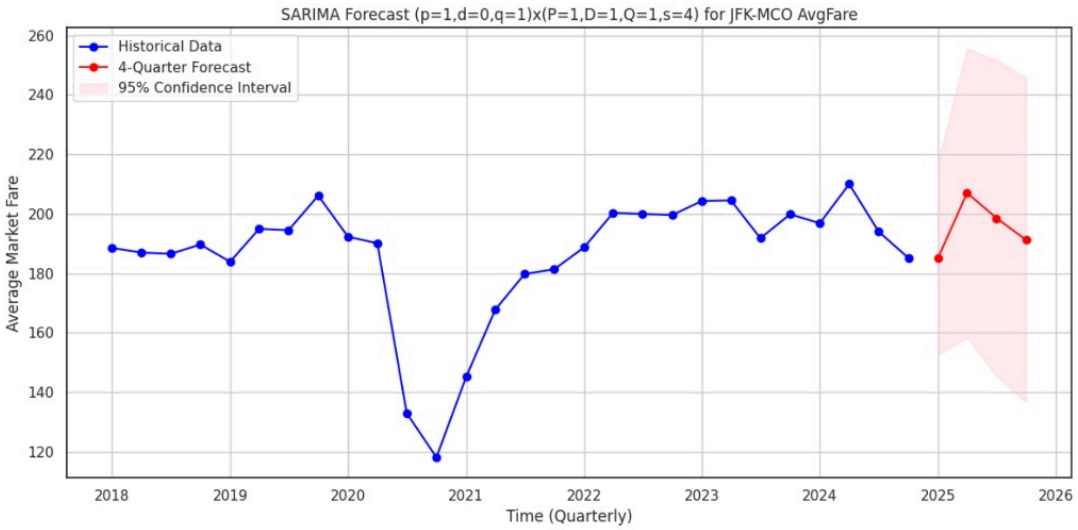


Figure 10: SARIMA forecast for JFK-MCO route fares. The red line indicates the forecast for the next 4 quarters, with the shaded region representing the 95% confidence interval.

4.6. Interactive Visualization Tool

To bridge the gap between static statistical analysis and actionable user insights, we developed an interactive web dashboard using **Plotly Dash** and **Folium**.

The dashboard is publicly accessible at: **https://sta141b-group2-flighttk.fly.dev/**

This interactive tool enables users to:

- **Select Origin-Destination routes**: Users can select any OD pair from the drop-down menu to plot the trend in fares per route.
- **Explore Historical Price Patterns (2018–2024)**: Average fares are shown on a dynamic, time-series chart on the dashboard to enable users to perceive long-run trends, seasonal movements, and disruptions such as the COVID-19 period.
- **Compare Multiple Routes**: Users may add multiple OD pairs to the plot, enabling side-by-side comparison of pricing behavior across different markets.
- **Interact With the Data**: Users can add more than one OD pair to the plot, allowing for side-by-side comparison of pricing behavior across different markets.

This interactive dashboard is a functional extension of our analyses; it also allows for transparency and reproducibility, and also provides future scalability when there is a need to monitor real-time fares or increase route coverage.

5. Summary / Conclusion

5.1. Synthesis of Findings

This study sought to demystify the drivers of domestic airfare and move beyond rough anecdotal "rules of thumb" toward precise, data-driven optimization. Our Multi Model Approach yielded four key insights:

1. **Structural Drivers**: The U.S. domestic market is **very concentrated** with 70% of the routes having an HHI above 2500. Our Random Forest model identified **Passenger Volume** and **Market Concentration** as the dominant predictors of ticket price, confirming that economies of scale-meaning high passenger density-strongly lowers per-seat costs ($r \approx -0.33$).

2. **Optimal Booking Strategy**: Analyzing a set of normalized price curves for millions of queries, we have identified a precise "Goldilocks Window" for buying tickets. The best time to purchase is consistently between **43 and 62 days** out, with the absolute best single day to buy being **50 days** out. Purchasing in the last week (0–7 days) results in a massive "procrastination penalty" of about **126%**.
3. **Pandemic Disruption**: Time series featured a very clear structural break due to COVID-19. Whereas routes for leisure have subsequently stabilized, business corridors such as ORD–DFW feature a significant inflationary "overshoot," consistent with a permanent change in pricing dynamics for corporate travel.
4. **Actionable Intelligence:** By embedding these statistical models in an **interactive dashboard**, we set up a platform for ongoing, long-term tracking of prices. This infrastructure not only lets us forecast concrete, model-backed forecasting, rather than just vague estimates, but also empowers users to better deal with the very complex volatility that characterizes modern airfare.

5.2. Limitations

Although our analysis provides sufficient statistical evidence, we faced some barriers to assessment that were related to the access and availability of data, coupled with the scope of the project:

- **Lack of Granular Historical Data**: The main issue here is that the resolution of the dataset DB1B aggregated data on a **quarterly** basis instead of providing specific dates of transactions. In addition, live APIs for flights, such as Amadeus, don't allow "back-tracing" to query historical prices. This means that in our time-series models, we used simulated day-of-year dates. Because of this smoothing effect, our model likely underestimates short-term micro-volatility like weekend-weekday premiums for booking.
- **Time Constraints & Scope**: Because of the strict 6-week project timeline, our research focused exclusively on major domestic hubs. If time permits, one could expand the analysis to international routes or even decentralized regional markets. This would be potentially useful to see if our findings regarding market concentration (HHI) hold in more recently deregulated international markets, or indeed under differing conditions of regulation.
- **Omitted Cost Variables:** Our regression models attribute price changes largely to time and demand. We did not include, however, explicit controls for **jet fuel prices**, which would be a massive operational cost driver. Much of the variance we attribute to "Time" is likely absorbing fluctuations in global energy prices.

5.3. Future Directions

In order to address the shortcomings of this research, three areas could be targeted in future research:

- **Constructing a Proprietary Dataset**: Moving on from aggregations on a quarterly basis, a mechanism for web scraping needs to be devised on a daily basis for a span of several years to come. This, in turn, would facilitate the identification of micro-trends such as prices spiking on the weekends, which have been overlooked owing to our current simulated timeliness.
- **Global Scope**: The conclusions from our research regarding HHI are confined to the U.S. domestic airline industry. We would be able to examine whether similar factors exist in a globally deregulated market, in which airline collaboration and subsidies have a greater impact, by extending our scope to **global routes**.
- **Advanced Deep Learning**: Although the SARIMA model offered a robust baseline, it is not very competent when it comes to highly fluctuating, non-linear behavior. The application of deep learning models, such as **Long Short-TermMemory (LSTM)** networks, is probably going to be more accurate in modeling the dependencies of travelers, resulting in more precise pricing.

References

Bibliography

[1] S. Borenstein and N. L. Rose, "Competition and Price Dispersion in the U.S. Airline Industry," *Journal of Political Economy*, vol. 102, no. 4, pp. 653–683, 1994.

[2] Dilwong, "Flight Prices Dataset." [Online]. Available: https://www.kaggle.com/datasets/dilwong/flightprices

[3] Amadeus IT Group, "Amadeus Flight Offers Search API." [Online]. Available: https://developers.amadeus.com/self-service/category/air/api-doc/flight-offers-search

List of Figures and Tables

List of Figures

List of Tables

Code Appendix

The following section contains the Python scripts used in our analysis and modeling pipeline. Highlighted portions indicate areas where external tools assisted in streamlining code execution and optimization.

Amadeus API Data Collection Code

The following Python code was used to collect real-time pricing data via the Amadeus Flight Offers Search API.

```
from datetime import date, timedelta
import pandas as pd
import time
import os
from amadeus import Client as AmadeusClient
from amadeus import ResponseError

# --- API Credentials ---
# Redacted for security
AMADEUS_CLIENT_ID = "YOUR_CLIENT_ID"
AMADEUS_CLIENT_SECRET = "YOUR_CLIENT_SECRET"

# --- Configuration ---
START_DAYS_AHEAD = 1
END_DAYS_AHEAD = 331
STEP = 1

ROUTES = [
    ("DEN", "JFK"), ("DFW", "ORD"), ("JFK", "MCO"),
    ("LAS", "LAX"), ("LAX", "SFO"), ("SEA", "SFO"),
]

# Standard distances for normalization
ROUTE_DISTANCES = {
    ("DEN", "JFK"): 1626, ("DFW", "ORD"): 802, ("JFK", "MCO"): 944,
    ("LAS", "LAX"): 236,  ("LAX", "SFO"): 337, ("SEA", "SFO"): 679
}

def get_distance(origin, dest):
    key = (origin, dest)
    rev_key = (dest, origin)
    return ROUTE_DISTANCES.get(key) or ROUTE_DISTANCES.get(rev_key)

def fetch_amadeus_flights(origin, dest, dep_date, days_ahead):
    """Query Amadeus API for lowest fare on a specific route/date."""
    print(f"   Searching {origin}->{dest} on {dep_date}...")
    try:
        response = amadeus.shopping.flight_offers_search.get(
            originLocationCode=origin,
            destinationLocationCode=dest,
            departureDate=dep_date,
            currencyCode="USD",
            adults=1
        )
        results = response.data
    except ResponseError as e:
        print("   API Error:", e)
```

```
        return []
    except Exception as e:
        print("   Other Error:", e)
        return []

    extracted_data = []
    for f in results:
        try:
            price = float(f.get("price", {}).get("total", 0))
            itinerary = f.get("itineraries", [{}])[0]
            duration_iso = itinerary.get("duration")
            segments = itinerary.get("segments", [])
            is_direct = 1 if len(segments) == 1 else 0
            airline_code = segments[0].get("carrierCode", "UNK")

            extracted_data.append({
                "query_date": date.today().isoformat(),
                "origin": origin,
                "dest": dest,
                "flight_date": dep_date,
                "days_ahead": days_ahead,
                "price": price,
                "airline": airline_code,
                "is_direct": is_direct,
                "MktDistance": get_distance(origin, dest)
            })
        except:
            continue
    return extracted_data

# --- Main Execution ---
try:
    amadeus = AmadeusClient(
        client_id=AMADEUS_CLIENT_ID,
        client_secret=AMADEUS_CLIENT_SECRET,
        environment="test" # Sandbox Environment
    )
    print("Client initialized.")
except Exception as e:
    print("Initialization failed:", e)
    exit()

all_rows = []
today = date.today()

for origin, dest in ROUTES:
    print(f"\n Route: {origin} -> {dest}")
    for days_ahead in range(START_DAYS_AHEAD, END_DAYS_AHEAD, STEP):
        dep_date = today + timedelta(days=days_ahead)
        flights = fetch_amadeus_flights(origin, dest, dep_date.isoformat(),
days_ahead)

        if flights:
            min_p = min(f["price"] for f in flights)
            print(f"      {len(flights)} offers. Min ${min_p}")
            all_rows.extend(flights)
```

```
        time.sleep(1) # Rate limit protection

# Save Results
if all_rows:
    df = pd.DataFrame(all_rows)
    filename = f"AMADEUS_FLIGHTS_{today.isoformat()}.csv"
    df.to_csv(filename, index=False)
    print("Data saved to", filename)
```

Data Cleaning Code

The following code extracted useful variables from the original csv files and converted the file into parquet format to save space.

```
import pandas as pd
import pdb
# Display all columns and rows
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", None)

# import DB1B Market data
df = pd.read_csv("/Users/wangchuyuan/Desktop/Fall25/STA141B/STA141B_Project/
T_DB1B_MARKET_2018_4.csv")
print("the original data", df.head(1))

df = df.rename(columns={
    'ITIN_ID': 'ItinID',
    'MARKET_COUPONS': 'MktCoupons',
    'YEAR': 'Year',
    'QUARTER': 'Quarter',
    'ORIGIN_AIRPORT_ID': 'OriginAirportID',
    'ORIGIN': 'Origin',
    'DEST_AIRPORT_ID': 'DestAirportID',
    'DEST': 'Dest',
    'TICKET_CARRIER': 'TkCarrier',
    'MARKET_FARE': 'MktFare',
    'MARKET_DISTANCE': 'MktDistance',
    'PASSENGERS': 'Passengers'
})


# list of desired Origin-Destination (OD) pairs
od_pairs = [
    ("LAX", "LAS"),
    ("DEN", "JFK"),
    ("ORD", "DFW"),
    ("LAX", "SFO"),
    ("JFK", "MCO"),
    ("SFO", "SEA")
]

# function to aggregate data by month for a given OD pair
def extract_od_flights(df, origin, dest):
    temp = df[(df["Origin"] == origin) & (df["Dest"] == dest)].copy()
    temp["origin"] = origin
    temp["dest"] = dest
    return temp
```

```
# loop through all OD pairs and aggregate
results = []
for origin, dest in od_pairs:
    cleaned = extract_od_flights(df, origin, dest)
    results.append(cleaned)
    # append OD pairs of both direction
    cleaned = extract_od_flights(df,dest,origin)
    results.append(cleaned)

# combine all OD pair data into a single tidy dataframe
tidy_df = pd.concat(results, ignore_index=True)


# sort by origin, destination, and month_date
tidy_df = tidy_df.sort_values(["origin", "dest"])

# display the first few rows and the total length of the data
print(tidy_df.head())
print("The length of our data is", len(tidy_df))

tidy_df.to_parquet("db1b_market_2018_4.parquet")
```

Analysis & Modeling Code

The following Python code was used to clean the DB1B data, generate rolling trends, calculate HHI, and train the Random Forest and SARIMA models.

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from statsmodels.tsa.statespace.sarimax import SARIMAX

# 1. Data Cleaning & Feature Engineering
#Remove outliers and simulate daily dates
def clean_and_process(df):
    # Filter Non-stop only
    df = df[df['MktCoupons'] == 1].copy()

    # Remove Outliers (IQR Method)
    Q1 = df['MktFare'].quantile(0.25)
    Q3 = df['MktFare'].quantile(0.75)
    IQR = Q3 - Q1
    df = df[(df['MktFare'] >= (Q1 - 1.5 * IQR)) &
            (df['MktFare'] <= (Q3 + 1.5 * IQR))]

    # Simulate Transaction Dates
    start_months = (df["Quarter"] - 1) * 3 + 1
```

```
    start_dates = pd.to_datetime(
        df["Year"].astype(str) + "-" + start_months.astype(str) + "-01"
    )
    random_days = np.random.randint(0, 90, size=len(df))
    df["random_date"] = start_dates + pd.to_timedelta(random_days, unit="D")

    return df

# 2. HHI Calculation
# Calculate Market Concentration.
def calculate_hhi(df):
    market_keys = ['Year', 'Quarter', 'Origin', 'Dest']

    # Market Totals
    df['Market_Total'] = df.groupby(market_keys)['Passengers'].transform('sum')

    # Carrier Shares
    carrier_shares = df.groupby(market_keys + ['TkCarrier'])
    ['Passengers'].sum().reset_index()
    carrier_shares = pd.merge(carrier_shares,
                              df[market_keys +
                              ['Market_Total']].drop_duplicates(),
                              on=market_keys)

    carrier_shares['Share_Sq'] = (carrier_shares['Passengers'] /
    carrier_shares['Market_Total']) ** 2

    # HHI Index (Sum of Squares * 10000)
    hhi = carrier_shares.groupby(market_keys)['Share_Sq'].sum().reset_index()
    hhi['HHI_Index'] = hhi['Share_Sq'] * 10000
    return hhi


#3. OLS Regression
def run_ols(df):
    formula = "MktFare ~ C(Airline) + HHI_Index + Passengers + MktDistance +
    Is_Covid"
    model = ols(formula, data=df).fit()
    print(model.summary())

# 4. Random Forest
def run_rf(df):
    features = ['HHI_Index', 'Passengers', 'MktDistance', 'Is_Covid']
    X = df[features]
    y = df['MktFare']
```

```
65
66      rf = RandomForestRegressor(n_estimators=100, random_state=42)
67      rf.fit(X, y)
68
69      # Extract Feature Importance
70      importances = pd.DataFrame({
71          'Feature': features,
72          'Importance': rf.feature_importances_
73      }).sort_values(by='Importance', ascending=False)
74      return importances
75
76
77  #5. SARIMA Forecasting
78  def run_sarima_forecast(df, route, periods=4):
79      """Forecast future prices for a specific route."""
80      route_data = df[df['OD_pair'] == route].set_index('TimePoint')
    ['AvgFare'].sort_index()
81      ts = route_data.resample('QS').mean().dropna()
82
83      # Fit Model (Seasonal Order 1,1,1,4 for Quarterly data)
84      model = SARIMAX(ts,
85                      order=(1, 1, 1),
86                      seasonal_order=(1, 1, 0, 4),
87                      enforce_stationarity=False)
88      results = model.fit(disp=False)
89
90      # Predict
91      forecast = results.get_forecast(steps=periods)
92      return forecast.predicted_mean, forecast.conf_int()
```

Dashboard Application Code

The following Python code serves as the entry point for the interactive Dash application, integrating Bootstrap for styling and registering callbacks for dynamic user exploration.

```python
from dash import Dash
import dash_bootstrap_components as dbc
from src.components.layout import create_layout
from src.callbacks import register_callbacks
def main() -> None:
    # Initialize Dash with Bootstrap theme
    app = Dash(
        __name__,
        external_stylesheets=[dbc.themes.FLATLY],
        suppress_callback_exceptions=True,
    )
    app.layout = create_layout(app)
    # Register interactive callbacks
    register_callbacks(app)
    app.run(debug=True)
if __name__ == "__main__":
    main()
```

Code Availability & Reproducibility

To ensure full transparency and reproducibility of our results, the complete source code for this project is hosted on GitHub.

Repository Link: https://github.com/Chuyuan-04/STA141B.git